Project 3: Musical Squares

Technical Reference Document

Nathaniel Lasher
EE526 - Wearable and Implantable Sensors / The University at Buffalo

Version 0.1 · November 01, 2026

Table of Contents

1 Project Overview 1
1.1 Purpose 1
1.2 Scope 1
1.3 Requirements 1
1.4 System Summary 1
1.5 Revision History 2
1.6 Testing 2
1.6.1 Debug Interface 2
2 Repository Layout 4
2.1 `TCS3472x.hpp` / `TCS3472x.cpp` 5
2.1.1 Functions 5
2.1.1.1 `initSensor()` 5
2.1.1.2 `writeReg()` 5
2.1.1.3 `readReg()` 5
2.1.1.4 `readColor()` 7
2.2 `synth.hpp` / `synth.cpp` 7
2.2.1 Functions 7
2.2.1.1 `synthInit()` 7
2.2.1.2 `synthUpdate()` 7
2.2.1.3 `advanceGlides()` 7
2.2.1.4 `stepFor()` 8
2.2.1.5 `clampFreq()` 8
2.2.1.6 `synthGlideTo()` 8
2.2.1.7 `synthGlideToHz()` 8
2.2.1.8 `synthPlay()` 8
2.2.1.9 `synthPlayHz()` 9
2.2.1.10 `synthStop()` 9
2.3 main.cpp 9
3 Hardware 10
3.1 Components 10
3.2 Pin Assignments 10
3.3 Power 10
4 Block and Schematic Diagrams 11
4.1 Block Diagram 11
4.2 Schematic 12
5 References 13

List of Figures

Figure 1 Left: Define macro for debug output routing. Right: Debug output example. 3
Figure 2 System block diagram. 11
Figure 3 Circuit schematic. 12

List of Listings

Listing 1 Repository layout. 4

1 Project Overview

1.1 Purpose

This project, realized for EE526 - Wearable and Implantable Sensors, is a project that I have had the idea for on my mind for a while. The purpose was to build your own unique smart toy device(s) using Arduino. I chose to try and work with sensors and protocols I haven't used before to add another layer to it. I was also facinated by the inner workings and split between photodiodes and LEDs.

1.2 Scope

The scope of this project includes the following aspects explicitly. This was more of a implementation checklist for organizing my thoughts at the start.

1.3 Requirements

ID	Requirement	Status
R1	~~RGB Color sensors will capture ambient light data~~	Done
R1.1	~~System will use the I2C protocol to communicate with the TCS3472x RGB Color sensor~~	Done
R1.2	System must react using RGB color sensors to colored tiles in real time	Planned
R1.3	~~System will be capable of toggling the built-in LED illumination of the color sensors on/off in software~~	Done
R2	~~System will output tones over DAC to 2 small speakers~~	Done
R2.1	System will map 8 tones to 8 individual colors	Planned
R2.2	System will be capable of playing a note for each of the 3 sensor tracks	Required
R2.2.1	System will be capable of playing multi-note chords	Planned
R2.3	~~System will be able to 'glyssando' linearly between each of the 8 planned tones~~	Done
R3	~~System will provide debug output over serial UART to a PC during normal operation~~	Done
R3.1	~~Debug output will include the initialization feedback~~	Done
R3.2	~~Debug output will include the readings from each of the RGB color sensors~~	Done
R3.3	~~Debug output will include the output note / DAC information~~	Done

1.4 System Summary

This section is a short description of how the pieces fit together. The System block diagram is shown in Figure 2 and the schematic detailed in Section 3. The main microcontroller is an Arduino Giga R1 Wifi driving the following; 2 5W speakers via DAC and a Class-D amplifier breakout for the audio output, and 3 TCS3472x RGB Color sensors via individual I2C busses. The Power supply supplies the Giga and then all other devices are powered on the 3.3V out pins provided by it. The reason for the overkill MCU is pinout real-estate. Each of the RGB color sensors have the same exact I2C address, making a single bus impossible. DAC cannot drive the speakers alone, hence the inclusion of the amplifier with a true system reference ground for interference mitigation.

1.5 Revision History

Version	Date	Changes
0.1	November 01, 2026	Initial draft

1.6 Testing

Throughout development I saved each inital stage of working behavior per component to create a directory of harware/software testbenches located in `Testbenches/`. See Section 2 for more information on the layout of this repository. For more information of the available debug terminal See Section 1.6.1. The purpose of each test is as follows:

Test Name (folder)	Description
`giga-blink/`	Ol' Relaiable. For getting the environment setup and flashing for the first time.
`rgb-sensor-debug`	Debug terminal available from this test forward. Configurable to test from a single RGB color sensor up to the hardware limit. Scan the I2C bus and log available devices as well as stream data from the found sensors. Meant to test sensor hardware and interface software.
`audio-debug`	Adds in the `synth` interface and allows for terminal commanded use to test the software module and speaker/DAC/Amp hardware.
`...`	More tests will be added as they are made.

1.6.1 Debug Interface

A simple debug terminal is exposed over a configurable port to provide a simple way to poke various system state. Echoing is implemented, though depending on the config it may be unreadable at times. Available commands are as follows:

Command	Description
`led`	Toggles the LEDs onboard each connected TCS3472x Sensor by pulling the `LED` pin low. Feedback is given on success with `led toggled`.
`stream`	Essentially the same as `led` but for the raw data stream from all connected RGB color sensors. Will toggle on command with successful operation denoted by `sensor debug stream <on/off>`.
`status`	Will print a table containing system state variables. May make this a live output in the future.
`play [0u-7u] [MIN_FREQf-MAX_FREQf]`	Exposes `synth` behavior. Tones 0-7 are avalable as well as frequencies from min to max (format 0 and 233.5 respectively). Successful operation echos the selected tone or freq. Track will glyssando on subsequent commands without a `stop`. All command act on track 0. Default argument is
`stop`	Silences track 0

Serial debug output can be routed according to the following code screenshot. Note that all raw sensor data is routed to `DEBUG_SERIAL`, while all terminal is routed through `DTERM_SERIAL`. See `main.ccp` for the code shown in Figure 1. Note also for any `Serial1` output that a USB-UART bridge is needed to capture Tx/Rx data and relay to a PC or similar terminal.

```
// This block steers where the output goes for debug functions
//   DEBUG = raw sensor stream
//   DTERM = boot and debug terminal info (typed commands and their replies)
//
//   1 = DEBUG on UART,  DTERM on UART   (both over D0/D1)
//   2 = DEBUG on USB,   DTERM on UART
//   3 = DEBUG on UART,  DTERM on USB
//   4 = DEBUG on USB,   DTERM on USB  (Both over usb)

#define OUTPUT_CONFIG 1

#if OUTPUT_CONFIG == 1
  #define DEBUG_SERIAL Serial1
  #define DTERM_SERIAL Serial1
  #define USES_UART 1
  #define USES_USB  0
#elif OUTPUT_CONFIG == 2
  #define DEBUG_SERIAL Serial
  #define DTERM_SERIAL Serial1
  #define USES_UART 1
  #define USES_USB  1
#elif OUTPUT_CONFIG == 3
  #define DEBUG_SERIAL Serial1
  #define DTERM_SERIAL Serial
  #define USES_UART 1
  #define USES_USB  1
#elif OUTPUT_CONFIG == 4
  #define DEBUG_SERIAL Serial
  #define DTERM_SERIAL Serial
  #define USES_UART 0
  #define USES_USB  1
  #error "OUTPUT_CONFIG must be 1, 2, 3 or 4"
#endif

#define BAUD_RATE 115200 // Serial UART baud rate to PC for debug output

const unsigned long DEBUG_INTERVAL = 100; // 100 ms
unsigned long lastOut = 0; // Marker for the time of previous output

bool streaming = false; // Off by default due to speaker interference
char cmdBuf[64]; // Serial input buffer
uint8_t cmdLen = 0; // Current length of buffer contents
```

```
LEFT C:2 R:0 G:1 B:0
LEFT C:2 R:0 G:1 B:0
LEFT C:2 R:0 G:1 B:0
LEFT C:2 R:0 G:1 B:0
LEFT C:2 R:0 G:1 B:0
LEFT C:2 R:0 G:1 B:0
LEFT C:2 R:0 G:1 B:0
LEFT C:2 R:0 G:1 B:0

sensor debug stream off
status
STATE
streaming=0
 led off
Sensor states:
LEFT OK
led
led toggled
led
led toggled
play
Now playing C4
stop
```

Figure 1: Left: Define macro for debug output routing. Right: Debug output example.

2 Repository Layout

This section provides a brief description of repository layout, source file roles and each function included in the project, each subsection corresponds to a separate source file. Each module is a header and implementation pair; `main.cpp` has no header. Below is a tree diagram of the repo layout and a table describing source roles.

```
./
├── Code/
│   └── Testbenches/
│       ├── audio-debug/
│       ├── giga-blink/
│       └── rgb-sensor-debug/
├── Documentation/
│   ├── Presentation/
│   └── Technical-reference/
│       ├── Images/
│       │   ├── debug-out-config.jpeg
│       │   ├── debug-out.jpg
│       │   ├── GigaR1Wifi-pinout.png
│       │   └── IMG_2811.jpeg
│       ├── tech-docs-ms.pdf
│       └── tech-docs-ms.typ
├── map-repo.sh
└── README.md
```

Listing 1: Repository layout.

Module	Responsibility
`TCS3472x.hpp`	Structure definitions, constants and function prototypes for .cpp counterpart
`TCS3472x.cpp`	Interface functions for I2C driven TCS3472x RGB color sensor
`main.cpp`	Entry point: `setup()` and `loop()`

2.1 `TCS3472x.hpp` / `TCS3472x.cpp`

This pair defines and implements the RGB color sensing. `N` Sensors are supported via a single enumeration `Sensor_Id` that (save for the debug output and I2C mappings) is completely configurable by adjusting that single point; For example running `rgb-sensor-debug/` with one sensor is possible by commenting out lines in the corresponding header file and functionality will follow suit. Sensors report ambient light and color data in real time via the data structures and interface funtions defined within this section

File	Contents
`TCS3472x.hpp`	Declares the `Color` data structure etc, function prototypes, and constants for RGB sensors
`TCS3472x.cpp`	Implements the 'driver' for these sensors. Including Read, Write, and Batched-Read color sampling from registers.
Depends on	`Wire.h`, `stdint.h`

2.1.1 Functions

The following functions exist as a simple hardware API for interfacing with the I2C driven configuration, control, and data registers on the TCS3472x RGB color sensor.

2.1.1.1 `initSensor()`

```
bool initSensor(TwoWire &bus);
```

Initializes the I2C bus and RGB sensor it is called upon. Call once for each connected bus-sensor pair from `setup()`. The enumeration in the paired .hpp file must be updated, but `NUM_SENSORS` can be used as the limit for a for loop iteration over this function to get the job done.

Defined in: `TCS3472x.cpp`

Returns: A `bool` whose value is `True` if init success; `False` otherwise. Note: expects the TCS3472x specifically and will return false for any other correctly connected sensor. See definition.

2.1.1.2 `writeReg()`

```
void writeReg(TwoWire &bus, uint8_t reg, uint8_t val);
```

Writes to a given register address given an I2C bus to preform the operation on. ADDR is the same for all sensors and unessecary to pass. Commands a static register address for a single write operation (of `val` to `reg`) using the effective message (`CMD` | `reg`).

Defined in: `TCS3472x.cpp`

Returns: Nothing.

2.1.1.3 `readReg()`

```
uint8_t readReg(TwoWire &bus, uint8_t reg);
```

Reads a given register address given an I2C bus to preform the operation on. ADDR is the same for all sensors and unessecary to pass. Commands a static register address for a single read operation using the effective message (`CMD` | `reg`) and subsequent bus read.

Defined in: `TCS3472x.cpp`

Returns: A `uint8_t` whose value represents the byte contents of a `reg`

2.1.1.4 `readColor()`

```
Color readColor(TwoWire &bus);
```

Writes to a given register address given an I2C bus to preform the operation on. ADDR is the same for all sensors and unessecary to pass. Commands a static register address for a single write operation using the effective message (`CMD` | `reg`).

Defined in: `TCS3472x.cpp`

Returns: A `Color` struct containing four usable `uint16_t` halfwords for each of the combined Low and High bytes for each of the colors' registers. See TCS3472x.hpp

2.2 `synth.hpp` / `synth.cpp`

These files provide the DAC interface functions required for tone generation. The pair provides methods to play, stop, blend, and ramp or 'glide' as referred to later tone on 1-2 speakers. Volume is adjustable in software via the `AMPLITUDE` constant. Dynamic range of the speakers is defined by `MAX_FREQ` and `MIN_FREQ`.

File	Contents
`synth.hpp`	Defines the Tone enum and Track structure as well as constants and function interfaecs. Configurability follows that of Section 2.1
`synth.cpp`	Source implementation for the above.
Depends on	`math.h`, `Arduino_AdvancedAnalog.h`, `Arduino.h`

2.2.1 Functions

2.2.1.1 `synthInit()`

```
void synthInit();
```

Initializes the DAC and DMA hardware required for tone generation. Responsible for the pre-calculation of the Sine approximation table `SINE_TABLE` and configuration of `SAMPLE_RATE`, `BUF_SAMPLES`, `BUF_COUNT`, and the analog resolution used during operation.

Defined in: `synth.cpp`

Returns: Nothing.

2.2.1.2 `synthUpdate()`

```
void synthUpdate();
```

Populates the DMA buffer responsible for feeding the DAC in real time. MUST be called imediately in every execution of loop; Loop should have no calls to `delay()`, See Section 2.3. When a DAC/DMA buffer becomes free, this function will populate it. Prior to population, it calls `advanceGlides()` to update each track's `phase`. See Section 2.2.1.3.

Defined in: `synth.cpp`

Returns: Nothing.

2.2.1.3 `advanceGlides()`

```
void advanceGlides();
```

On call, moves every track that is currently changing tone or 'gliding' one `step` closer to it's `target_hz` by incrementing `hz+=hz_per_buffer`. When a track has reached it's target frequency (`hz == target_hz`), this function returns with no effect. Rate of change of `hz` is defined by the glide provoking call from `GlideTo` functions. See Section 2.2.1.6 and Section 2.2.1.7.

Defined in: `synth.cpp`

Returns: Nothing.

2.2.1.4 `stepFor()`

```
float stepFor(float hz);
```

Simply calculates the `step` field value. This value represents the distance of each 'step' through the `SINE_TABLE`. This value is used in `synthUpdate()` to increment `phase`. See Section 2.2.1.2.

Defined in: `synth.cpp`

Returns: `float` representing the change in `phase` index (into `SINE_TABLE`) to be applied at next update

2.2.1.5 `clampFreq()`

```
float clampFreq(float hz);
```

Clamps the value of `hz` to the dynamic range of the installed speakers as set by the `MIN_FREQ` and `MAX_FREQ` constants.

Defined in: `synth.cpp`

Returns: Returns `MIN_FREQ` if `hz < MIN_FREQ`, `MAX_FREQ` if `hz > MAX_FREQ`, and `hz` otherwise.

2.2.1.6 `synthGlideTo()`

```
void synthGlideTo(uint8_t track, Tone tone, uint32_t ms);
```

Converts a `Tone` into hz via the enumeration value passed into `tone`, then calls `synthGlideToHz()` on this value, applying it to `track` smoothly over a period of `ms` milliseconds. See Section 2.2.1.7.

Defined in: `synth.cpp`

Returns: Nothing.

2.2.1.7 `synthGlideToHz()`

```
void synthGlideToHz(uint8_t track, float hz, uint32_t ms);
```

When called on an active `track` with a `hz` value different to `track.hz`, configures the track to shift it's tone to the new target frequency over a period of `ms` milliseconds. Responsible for calculating the `hz_per_buffer` field. Calls `synthPlayHz()` on an inactive or non-gliding track. Can be called with an arbitrary frequency within the dynamic range defined by `MIN_FREQ` and `MAX_FREQ`. See Section 2.2.1.5 and Section 2.2.1.9.

Defined in: `synth.cpp`

Returns: Nothing.

2.2.1.8 `synthPlay()`

```
void synthPlay(uint8_t track, Tone tone);
```

Converts a `Tone` into hz via the enumeration value passed into `tone`, then calls `synthPlayHz()` on this value, applying it to `track`. See Section 2.2.1.9.

Defined in: `synth.cpp`

Returns: Nothing.

2.2.1.9 `synthPlayHz()`

```
void synthPlayHz(uint8_t track, float hz);
```

When called on an active `track` with a `hz` value different to `track.hz`, configures the track to imediately change it's tone to the new frequency. Will activate non-active tracks automatically. Can be called with an arbitrary frequency within the dynamic range defined by `MIN_FREQ` and `MAX_FREQ`. See Section 2.2.1.5.

Defined in: `synth.cpp`

Returns: Nothing.

2.2.1.10 `synthStop()`

```
void synthStop(uint8_t track);
```

Imediately stops a tracks playback by setting `active` to `false`.

Defined in: `synth.cpp`

Returns: Nothing.

2.3 main.cpp

3 Hardware

3.1 Components

Part	Description	Qty	Interface	Cost
Arduino Giga R1 Wifi	STM32 based microcontroller with 3 I2C lanes and DAC.	1	USB, JTAG, UART	$72.82
TCS-34725	RGB Color/Light Sensor	1	I2C	$18.02
PAM8403	Dual Channel Class-D Amplifier Break-out Board	1	DAC	$5.99
Speaker	Generic Amazon 5W 8 Ohm speaker	2	Analog	$7.85

3.2 Pin Assignments

Pin	Signal	Connected to
`D2`	`SDA`	TODO
`D3`	`SCL`	TODO

3.3 Power

TODO: Supply voltage, regulators, and estimated current draw. Currently wall power.

4 Block and Schematic Diagrams

4.1 Block Diagram

Initial hardware planning block diagram.

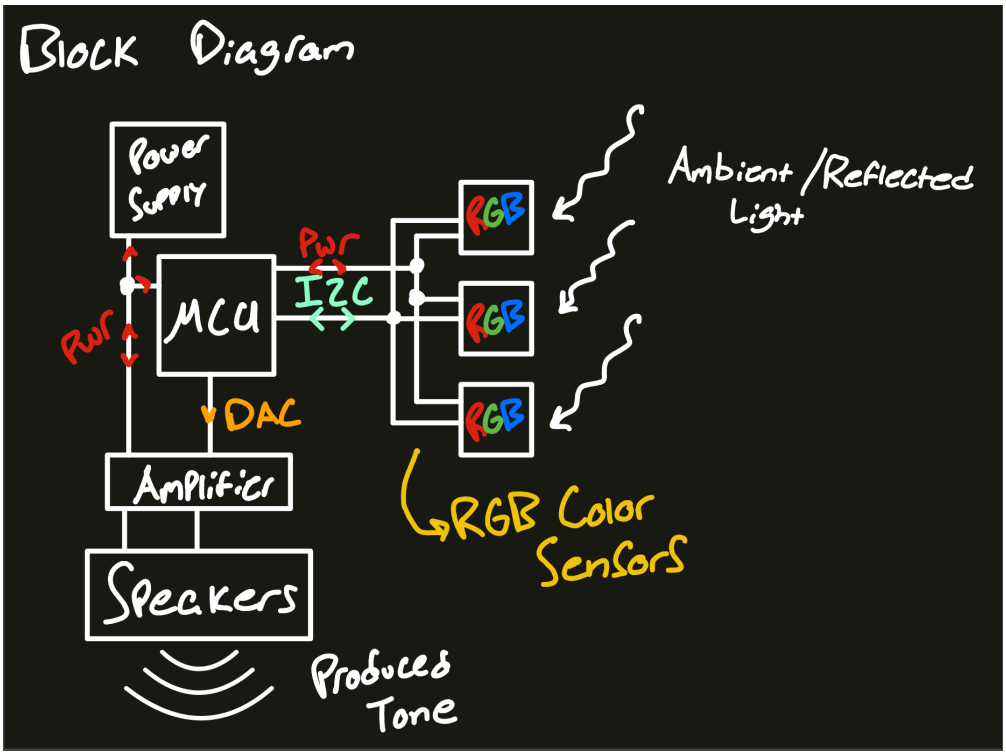


Figure 2: System block diagram.

4.2 Schematic

Hardware schematic showing the pin connections, circuit components and power routing.



Figure 3: Circuit schematic.

5 References

TODO: This section should include a full list of references. Note that I setup typst, emacs, and platformio for the first time with this project so I'm including that material.

— Typst (official) — https://typst.app/docs/ https://typst.app/docs/reference/ https://typst.app/docs/tutorial/ https://typst.app/docs/reference/syntax/ https://typst.app/docs/reference/scripting/ https://typst.app/docs/reference/styling/ https://typst.app/docs/reference/model/figure/ https://typst.app/docs/reference/model/outline/ https://typst.app/docs/reference/model/table/ https://typst.app/docs/reference/text/raw/ https://typst.app/docs/reference/layout/page/ https://typst.app/docs/reference/data-loading/read/ https://typst.app/universe/ https://github.com/typst/typst/releases

— Typst (community / editor tooling) — https://sitandr.github.io/typst-examples-book/ https://github.com/Ziqi-Yang/typst-ts-mode https://zff.dev/meow_king/typst-ts-mode/wiki/Installation https://github.com/Myriad-Dreamin/tinymist https://github.com/uben0/tree-sitter-typst

— PlatformIO — https://docs.platformio.org/ https://docs.platformio.org/en/stable/integration/ide/emacs.html https://github.com/zachmassia/platformio-mode https://bit.ly/package-manager-ioerror https://raw.githubusercontent.com/platformio/platformio-core/develop/platformio/assets/system/99-platformio-udev.rules https://community.platformio.org/t/arduino-giga-r1-wifi-board-configuration/34409 https://community.platformio.org/t/giga-dfu-issues/41982

— Emacs — https://www.gnu.org/software/emacs/manual/ https://www.gnu.org/software/emacs/manual/html_node/use-package/ https://www.gnu.org/software/emacs/manual/html_mono/eglot.html https://www.gnu.org/software/emacs/manual/html_node/elisp/index.html https://www.gnu.org/software/auctex/manual/auctex/index.html https://melpa.org/

— Language servers and language modes — https://clangd.llvm.org/ https://pkg.go.dev/golang.org/x/tools/gopls https://github.com/dominikh/go-mode.el https://github.com/python-lsp/python-lsp-server https://github.com/chipsalliance/verible https://github.com/latex-lsp/texlab

— RGB Color Sensor documentation TCS3472X datasheet (TAOS135, August 2012), https://cdn-shop.adafruit.com/datasheets/TCS3472x.pdf